

2.

```
#include <stdio.h>
#include <omp.h>
int main() {
    int num_iterations;
    printf("Enter number of iterations: ");
    scanf("%d", &num_iterations);
    omp_set_schedule(omp_sched_static, 2);
    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        #pragma omp for schedule(runtime)
        for (int i = 0; i < num_iterations; i++) {
            printf("Thread %d: iteration %d\n", thread_id, i);
        }
    }
    return 0;}
```

3.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int is_prime(int num)
{
    if (num <= 1)
        return 0;
    for (int i = 2; i * i <= num; i++)
    {
        if (num % i == 0)
            return 0;
    }
    return 1;
}
int main()
{
    int n;
    printf("Enter the value of n: ");
    scanf("%d", &n);
    double start, end;
    start = omp_get_wtime();
    printf("Serial Prime Numbers: ");
    for (int i = 1; i <= n; i++)
    {
        if (is_prime(i))
        {
            printf("%d ", i);
        }
    }
    printf("\n");
    end = omp_get_wtime();
    printf("Serial Execution Time %f seconds\n", end - start);
}
```

```

start = omp_get_wtime();
printf("Parallel Prime Numbers: ");
#pragma omp parallel for schedule(dynamic)
for (int i = 1; i <= n; i++)
{
    if (is_prime(i))
    {
        #pragma omp critical
        {
            printf("%d ", i);
        }
    }
}
printf("\n");
end = omp_get_wtime();
printf("Parallel Execution Time %f seconds\n", end - start);
return 0;
}

```

4.

```

#include <stdio.h>
#include <omp.h>
int fibonacci(int n){
    int x,y;
    if(n<=1)
        return n;
    #pragma omp task shared(x)
    x=fibonacci(n-1);
    #pragma omp task shared(y)
    y=fibonacci(n-2);
    #pragma omp taskwait
    return x+y;
}
int main(){
    int n,result;
    printf("Enter value of n (Fibonacci number to calculate): ");
    scanf("%d",&n);
    double start_time=omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        result=fibonacci(n);
    }
    double end_time=omp_get_wtime();
    printf("Fibonacci number is %d\n",result);
    printf("Time taken is %f seconds\n",end_time-start_time);
    return 0;
}

```

5.

```

#include <stdio.h>

```

```

#include <mpi.h>
int main(int argc, char *argv[]){
    int rank, size;
    int number;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size < 2){
        if(rank == 0)
            printf("Please run with at least 2 processes\n");
    }
    if(rank == 0){
        number = 100;
        MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Process 0 sent number %d to Process 1\n", number);
    }
    if(rank == 1){
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from Process 0\n", number);
    }
    MPI_Finalize();
    return 0;
}

```

6.

```

#include <stdio.h>
#include <mpi.h>
int main(int argc, char *argv[]){
    int rank, size, data;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(rank == 0){
        printf("Enter a value of broadcast: ");
        scanf("%d", &data);
    }
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received data = %d\n", rank, data);
    MPI_Finalize();
    return 0;
}

```

7.

```

#include <stdio.h>
#include <mpi.h>
int main(int argc, char *argv[]){
    int rank, data = 200;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if(rank == 0){
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
}

```

```

MPI_Recv(&data,1,MPI_INT,1,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
printf("Process 0 completed communication.\n");
}
else if(rank==1){
MPI_Send(&data,1,MPI_INT,0,0,MPI_COMM_WORLD);
MPI_Recv(&data,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
printf("Process 1 completed communication.\n");
}
MPI_Finalize();
return 0;
}

```

8.

```

#include <stdio.h>
#include <mpi.h>
int main(int argc,char *argv[]){
int rank,size;
int data[8],recv_data;
int gathered_data[8];
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);
if(size!=8){
if(rank==0){
printf("Please run the program with 8 processes\n");
}
MPI_Finalize();
return 0;
}
if(rank==0){
for(int i=0;i<8;i++){
data[i]=i+1;
}
}
MPI_Scatter(data,1,MPI_INT,&recv_data,1,MPI_INT,0,MPI_COMM_WORLD);
recv_data*=10;
MPI_Gather(&recv_data,1,MPI_INT,gathered_data,1,MPI_INT,0,MPI_COMM_WORLD);
if(rank==0){
printf("Gathered data after processing:\n");
for(int i=0;i<8;i++){
printf("%d ",gathered_data[i]);
}
printf("\n");
}
MPI_Finalize();
return 0;
}

```